

Clase 127 — Heap: use-after-free y double free

Parte: 5 — Explotación de sistemas y binarios · Fuente: *The Shellcoder's Handbook* · how2heap (Shellphish) 🕒 Duración estimada: 140 min · Nivel: Experto

🎯 Objetivo

Explotar dos de las corrupciones de heap más frecuentes: **use-after-free (UAF)**, donde se usa un puntero a memoria ya liberada, y **double free**, donde se libera dos veces el mismo chunk. Verás cómo el UAF permite leer/escribir datos ajenos y secuestrar punteros de función, y cómo un double free en tcache (**tcache poisoning**) logra escritura arbitraria para redirigir `malloc` a una dirección elegida.

⚠️ **Ética:** solo en laboratorio propio o CTF autorizado.

📊 Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Explicar** las causas de UAF y double free y su impacto.
- 2. **Aprovechar** un UAF para leer/escribir sobre un objeto reasignado.
- 3. **Realizar** tcache poisoning a partir de un double free.
- 4. **Redirigir** `malloc` a una dirección arbitraria (p. ej. `__free_hook` /GOT).
- 5. **Detectar** estos bugs con ASan.

📖 Temas

#	Tema	Por qué importa
1	Puntero colgante (dangling)	Raíz del UAF
2	Reasignación del chunk	Cómo se solapan objetos
3	Secuestro de vtable/punteros	UAF → control de flujo
4	Double free en tcache	Base del poisoning
5	tcache key (mitigación)	Detección de double free en glibc ≥2.29
6	tcache poisoning	Escritura del <code>next</code> → malloc arbitrario
7	Objetivos: <code>__free_hook</code> , GOT	Convertir en RCE
8	Detección con ASan	Cazar el bug en desarrollo

Definiciones y características

- **Use-after-free:** uso de memoria tras `free`. *Clave:* CWE-416; si el chunk se reasigna, escribes sobre otro objeto.
- **Puntero colgante:** referencia que sobrevive al `free`. *Clave:* no ponerlo a `NULL` es la causa típica.
- **Double free:** liberar dos veces el mismo puntero. *Clave:* corrompe la lista del bin, permitiendo devolver el mismo chunk dos veces.
- **tcache poisoning:** sobrescribir el puntero `next` de un chunk en tcache. *Clave:* el siguiente `malloc` de ese tamaño devuelve la dirección que elijas.
- **tcache key:** campo que glibc moderna usa para detectar double free en tcache. *Clave:* hay que falsearlo o usar otra ruta.
- **__free_hook / __malloc_hook:** punteros de función históricos usados como objetivo de escritura. *Clave:* eliminados en glibc ≥2.34; hoy se apunta a GOT/estructuras alternativas.

Herramientas y preparación

```
pip install pwntools
# ASan para detección:
gcc -fsanitize=address -g uaf.c -o uaf_asan
ldd --version | head -1      # conocer la versión de glibc y sus mitigaciones
```

Laboratorio guiado

Entorno propio.

1. UAF — programa `uaf.c`:

```
c #include <stdio.h> #include <stdlib.h> #include <string.h> int main(){ char *a =
malloc(0x40); free(a); // a queda colgante char *b = malloc(0x40); // reutiliza el mismo chunk
strcpy(b, "datos de b"); printf("a ahora lee: %s\n", a); // UAF: a y b son la misma memoria }
```

Ejecuta y observa que `a` "ve" lo que escribió `b`.

2. Detecta el bug con ASan: `./uaf_asan` reporta `heap-use-after-free` con backtrace.

3. Double free / tcache poisoning en pwndbg (glibc que lo permita):

- Reserva dos chunks del mismo tamaño, libéralos y provoca el double free controlado.
- Sobrescribe el `next` del chunk en tcache con la dirección objetivo:

```
python from pwn import * # ...interacción con el binario de reto... # tras el double free,
escribimos el next del chunk liberado: edit(idx, p64(target_addr)) # target_addr =
&__free_hook o entrada GOT malloc(size) # saca el chunk falso al frente malloc(size) # este
malloc devuelve target_addr
```

4. Escribe en el chunk devuelto para colocar `system` en `__free_hook` (o el equivalente moderno) y dispara un `free` sobre un chunk que contenga `"/bin/sh"`.

5. Verifica en GDB que `malloc` devolvió tu dirección objetivo (`vis_heap_chunks` , `bins`).
6. Comenta cómo la `tcache key` de glibc moderna obligaría a un paso extra para el double free.

Ejercicios

1. Corrige el `uaf.c` poniendo el puntero a `NULL` tras `free` y confirma con ASan.
2. Explica por qué el chunk reasignado solapa los dos punteros.
3. Realiza tcache poisoning apuntando a una variable global conocida.
4. Investiga la `tcache key` y cómo detecta el double free.
5. Enumera objetivos de escritura viables en tu versión de glibc.
6. Compara la salida de ASan con la de Valgrind para el mismo UAF.

Reto verificable

En un binario de reto con UAF/double free, logra que un `malloc` devuelva una dirección que tú elijas mediante tcache poisoning.

Criterio de aceptación: demuestras en GDB que un `malloc` retorna la dirección objetivo, y (si el reto lo permite) obtienes ejecución de código o lectura de la flag.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>free(): double free detected in tcache 2</code>	tcache key activa; falséala o usa fastbin
El poisoning no cambia malloc	El <code>next</code> mal alineado o tamaño de bin incorrecto
Crash al liberar chunk falso	Metadatos inválidos; ajusta <code>size</code> y alineación
<code>__free_hook</code> no existe	glibc ≥ 2.34 ; apunta a GOT/estructuras alternativas
ASan no detecta nada	No compilaste con <code>-fsanitize=address</code>

Preguntas frecuentes

? ¿UAF siempre da RCE? No siempre; depende de si el objeto reasignado contiene punteros de función o datos sensibles.

? ¿Sigue funcionando el ataque a `__free_hook`? No en glibc ≥ 2.34 (los hooks fueron eliminados); hay que buscar objetivos modernos.

? ¿Cómo evito estos bugs al programar? Pon punteros a `NULL` tras `free` , usa smart pointers en C++ y compila con ASan en CI.

Referencias

- CWE-416: Use After Free — <https://cwe.mitre.org/data/definitions/416.html>
- how2heap (Shellphish) — <https://github.com/shellphish/how2heap>
- The Shellcoder's Handbook, cap. de heap. Wiley.

- AddressSanitizer — <https://github.com/google/sanitizers/wiki/AddressSanitizer>



Material descargable

- [Guía en PDF](#) — versión imprimible de esta clase.
- [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Siguiente clase

[Clase 128 - Integer overflows y errores aritmeticos](#)